NASCAR for CubeSats: CubeSat Racing Ring Deployment and Control

Ian M. Faber*
University of Colorado Boulder, Boulder, CO, 80303

The concept of CubeSat racing is revisited with an emphasis on race course formation control. A formation deployment and stowing sequence is proposed to transition the race course from a fuel-saving lead-follower formation to the full race course and back again. Control effort throughout the sequence is analyzed and the controllers implemented are assessed, and suggestions for further refinement of the sequence are provided.

I. Nomenclature

A	=	Satellite cross-sectional area
$[A(\mathbf{oe})]$	=	Natural orbit element evolution matrix
a	=	Semi-major axis
$[B(\mathbf{oe})]$	=	orbit element control to output matrix
C_d	=	Satellite coefficient of drag
e	=	Orbit eccentricity
f	=	Orbit true anomaly
$\dot{f}$	=	Orbit true anomaly rate of change
$\mathbf{f}$	=	Natural orbit acceleration function
h	=	Orbit specific angular momentum
i	=	Orbit inclination
J_2	=	Planet J_2 gravitational constant
$[K]$	=	Gain matrix
m	=	Satellite mass
M	=	Orbit mean anomaly
μ	=	Planet gravitational parameter
$\mathcal{N}$	=	Inertial frame
$[\mathcal{N}O]$	=	DCM to Inertial frame from Orbit frame
$\hat{\mathbf{o}}$	=	Orbit frame unit vector
O	=	Orbit/Hill frame
$\mathbf{oe}_d$	=	Deputy orbital elements vector
$\mathbf{oe}_r$	=	Reference orbital elements vector
$\Delta\mathbf{oe}$	=	Orbit element difference vector
Ω	=	Orbit RAAN
ω	=	Orbit argument of periapsis
$\omega_{O/\mathcal{N}}$	=	Angular velocity of frame O with respect to frame $\mathcal{N}$
p	=	Orbit semi-latus rectum
ϕ	=	Orbit planetary latitude
R	=	Gravitational potential function
$\mathbf{r}$	=	Position vector
$\dot{\mathbf{r}}$	=	Position vector rate of change
$\mathbf{r}_r$	=	Reference cartesian position vector
$\mathbf{r}_d$	=	Deputy cartesian position vector
$\mathbf{r}_c$	=	Chief cartesian position vector
${}^{\mathcal{N}}\mathbf{r}$	=	Position vector in the Inertial frame
$\Delta\mathbf{r}$	=	Cartesian position error vector
$\Delta\dot{\mathbf{r}}$	=	Cartesian velocity error vector
r	=	Orbit radius
r_{eq}	=	Equivalent planetary radius
$\dot{r}$	=	Orbit radius rate of change
$\boldsymbol{\rho}$	=	Relative position vector
ρ	=	Atmospheric density
ρ_0	=	Reference atmospheric density
$\mathbf{u}$	=	Control acceleration
$\mathbf{u}_r$	=	Feedforward acceleration
$\mathbf{v}$	=	Velocity vector
$\mathbf{v}_d$	=	Deputy cartesian velocity vector
$\mathbf{v}_r$	=	Reference cartesian velocity vector
v	=	Satellite speed
V	=	Lyapunov function
$\dot{V}$	=	Lyapunov function rate of change
x	=	Radial relative separation
$\dot{x}$	=	Radial relative speed
y	=	Along track relative separation
$\dot{y}$	=	Along track relative speed
z	=	Cross track relative separation
$\dot{z}$	=	Cross track relative speed

*Graduate Student, Ann & H.J. Smead Aerospace Engineering Sciences, 3775 Discovery Dr, Boulder, CO 80303, AIAA student (1600652)

II. Introduction

In a previous project, the idea of CubeSat racing was introduced from an optimal control perspective [1]. The idea was simple: Given a randomized course of waypoint rings and a randomized starting position, compute the optimal trajectory between waypoints that minimizes the time taken for a CubeSat to traverse the entire course while also minimizing the distance to the center of each ring. It was found that primer vector theory [2] in context of the linearized Clohessy-Wiltshire Hill (CWH) equations could be leveraged to generate trajectories for each CubeSat and accomplish the goal.

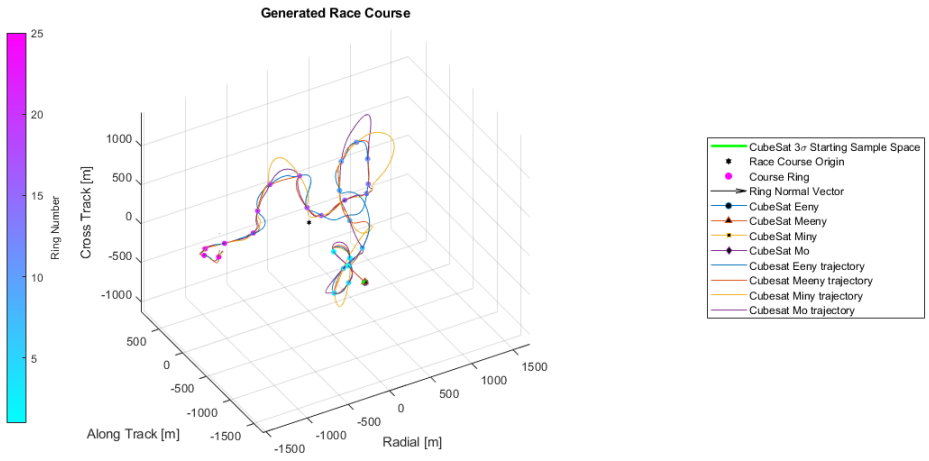

Fig. 1 Example CubeSat Race

However, a critical assumption made in the project was that the rings themselves are not governed by any relative equations of motion, such as the CWH equations. This is not very realistic, as in reality these rings are also in orbit, and there are a number of orbit perturbations present that could change the geometry of the race course over time. To expand upon this, a followup project explored the impact of the full, nonlinear relative equations of motion for spacecraft formations on the race course geometry, in addition to the impact of J_2 nonuniform gravity and atmospheric drag on the resulting trajectories [3]. In summary, it was found that the orbit perturbations and relative equations of motion have a profound effect on the evolution of the race course geometry.

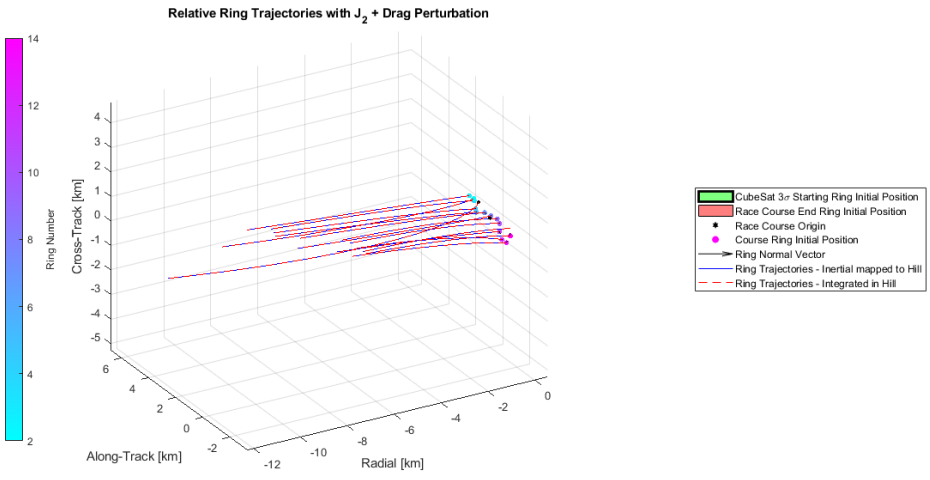

Fig. 2 Race Course Evolution with Orbit Perturbations

In context of the race course evolution shown in Figure 2, it was proposed in [3] that a formation control strategy was needed to maintain the correct race course geometry in the presence of perturbations. However, it was also suggested that constant race course formation control would be expensive, so the idea of a ring deployment and stowing sequence from a lead-follower formation to the full race course formation and back again was proposed. This project expands upon that idea and implements an inertial cartesian feedback control law for the race course formation and an orbit element difference feedback control law for the lead-follower formation.

III. Task 0: Randomized Race Course Generation

The first step of this project was to generate CubeSat race courses at random. Luckily, [1] already accomplished this, and the method is repeated here for context. For this project, the size of the end ring was increased for visibility.

A race course is made up of a sequence of rings that abide by the following requirements:

1) Individual race course rings shall have semi-major and semi-minor axes uniformly distributed between $[1, 5]$ m.
2) Rings shall be spaced apart with uniformly distributed relative azimuth angles $\theta_r \in [-90, 90]^\circ$, relative elevation angles $\phi_r \in [-90, 90]^\circ$, and inter-ring distances $d \in [300, 350]$ m.
    a) The relative azimuth angle is defined relative to the last ring's normal vector in the xy plane, and the relative elevation angle is defined as the angle with respect to the previous ring's normal vector in that ring's z direction.
    b) For example, a relative azimuth and elevation angle of [0, 0]° would point parallel to the last ring. Similarly, an angle combination of [45, -45]° would point to the right and down of the last ring's normal vector.
    c) For plotting, relative azimuth and elevation angles are converted to absolute angles with respect to the race course x axis and race course xy plane, respectively.
3) The first race course ring will be aligned along the course +y axis.
4) Each race course shall consist of 15 rings.

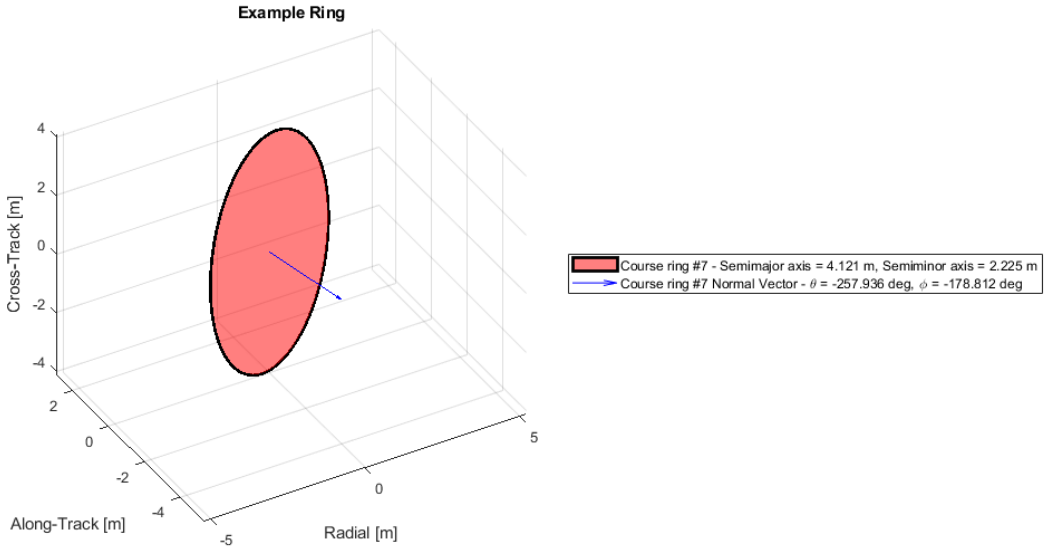

Fig. 3 Example Race Course Ring

Once all rings are generated, the centroid of all the rings is calculated and set as the origin of the race course, which will act as the deputy "spacecraft" for this project. Since it is a point and not a spacecraft by itself, the race course origin does not need to have perturbations applied to it and can follow a fully Keplerian orbit. This is an assumption that is applied and then relaxed between controllers, depending on the objective of the controller. Once all rings have been

generated and the race course origin has been assigned, the result is a randomized race course as seen in Figure 4. For the purposes of this project, MATLAB's `rng()` method was given a seed of 3 for repeatability, but the race course can be completely randomly generated with `rng('shuffle')` instead.

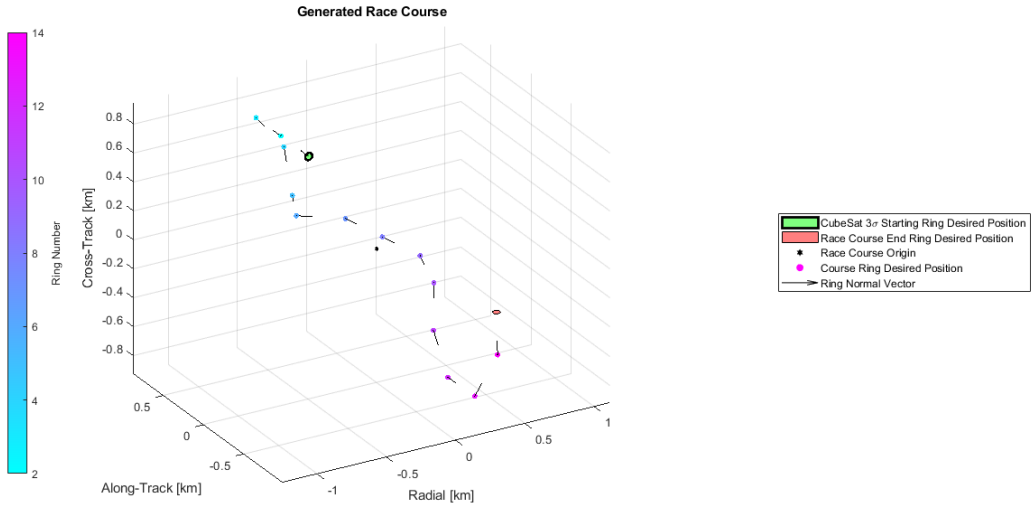

Fig. 4 Example Randomly Generated Race Course

IV. Task 1: Formation Control Law Development

For this project, two control laws are needed: One for the active race course formation, and one for the idle lead-follower formation.

A. Race Course Formation Control

The goal of the race course formation control law is to deploy the rings quickly and then maintain the ring positions over the duration of a CubeSat race. Thus, a controller that can be easily tuned for response time and with sufficient robustness to perturbations is needed. To that end, an inertial cartesian feedback control law was chosen.

We know the inertial equations of motion for a satellite orbiting a celestial body are as follows [4]:

$$\ddot{\mathbf{r}} = \mathbf{f}(\mathbf{r}, \mathbf{v}) + \mathbf{u} \tag{1}$$

where $\mathbf{f}(\mathbf{r}, \mathbf{v})$ is a function returning the natural acceleration vector from the current state and $\mathbf{u}$ is a control acceleration applied to the spacecraft. In the case of the inertial controller, which has to compensate for J_2 and drag, $\mathbf{f}$ can be written as

$$\begin{aligned}\mathbf{f}(\mathbf{r}, \mathbf{v}) &= -\frac{\mu}{r^3}{}^{\mathcal{N}}\mathbf{r} + \frac{\mu J_2 r_{eq}^2}{r^5}\underbrace{\begin{bmatrix} \frac{15}{2}\sin^2\phi - \frac{3}{2} & 0 & 0 \\ 0 & \frac{15}{2}\sin^2\phi - \frac{3}{2} & 0 \\ 0 & 0 & \frac{15}{2}\sin^2\phi - \frac{9}{2} \end{bmatrix}}_{[A_{J_2}]}{}^{\mathcal{N}}\mathbf{r} - \frac{1}{2}\rho\frac{C_d A}{m}v_{rel}{}^{\mathcal{N}}\mathbf{v}_{rel} \\ &= -\frac{\mu}{r^3}{}^{\mathcal{N}}\mathbf{r} + \frac{\mu J_2 r_{eq}^2}{r^5}\left[A_{J_2}\right]{}^{\mathcal{N}}\mathbf{r} - \frac{1}{2}\rho\frac{C_d A}{m}v_{rel}{}^{\mathcal{N}}\mathbf{v}_{rel}\end{aligned} \tag{2}$$

To derive the race course formation control law, we will utilize Lyapunov's Direct Method. We can define error vectors $\Delta\mathbf{r} = \mathbf{r}_d - \mathbf{r}_r$ and $\Delta\dot{\mathbf{r}} = \mathbf{v}_d - \mathbf{v}_r$ and use them to define the following positive definite, radially unbounded candidate Lyapunov function:

$$V(\Delta\mathbf{r}, \Delta\dot{\mathbf{r}}) = \frac{1}{2}\Delta\dot{\mathbf{r}}^\intercal \Delta\dot{\mathbf{r}} + \frac{1}{2}\Delta\mathbf{r}^\intercal \left[K_1\right] \Delta\mathbf{r} \tag{3}$$

where $[K_1]$ is a positive definite matrix. Taking the derivative of the candidate Lyapunov function in Equation 3 results in the following expression:

$$\begin{aligned}
\dot{V} &= \Delta\dot{\mathbf{r}}^\intercal \Delta\ddot{\mathbf{r}} + \Delta\dot{\mathbf{r}}^\intercal \left[K_1\right] \Delta\mathbf{r} \\
&= \Delta\dot{\mathbf{r}}^\intercal \left(\ddot{\mathbf{r}}_d - \ddot{\mathbf{r}}_r + \left[K_1\right] \Delta\mathbf{r}\right) \\
&= \Delta\dot{\mathbf{r}}^\intercal \left(\mathbf{f}(\mathbf{r}_d, \mathbf{v}_d) + \mathbf{u} - \mathbf{f}(\mathbf{r}_r, \mathbf{v}_r) - \mathbf{u}_r + \left[K_1\right] \Delta\mathbf{r}\right)
\end{aligned} \tag{4}$$

To be a valid Lyapunov function, we need $\dot{V}$ to be negative semidefinite. We can force this by setting Equation 4 equal to the negative semidefinite expression $\dot{V}(\Delta\dot{\mathbf{r}}) = -\Delta\dot{\mathbf{r}}^\intercal \left[K_2\right] \Delta\dot{\mathbf{r}}$, where $[K_2]$ is a positive definite matrix. Equating and solving for $\mathbf{u}$,

$$\begin{aligned}
\mathbf{u} + \mathbf{f}(\mathbf{r}_d, \mathbf{v}_d) - \mathbf{f}(\mathbf{r}_r, \mathbf{v}_r) - \mathbf{u}_r + \left[K_1\right] \Delta\mathbf{r} &= -\left[K_2\right] \Delta\dot{\mathbf{r}} \\
&\downarrow \\
\mathbf{u} &= -\left(\mathbf{f}(\mathbf{r}_d, \mathbf{v}_d) - \mathbf{f}(\mathbf{r}_r, \mathbf{v}_r)\right) - \left[K_1\right] \Delta\mathbf{r} - \left[K_2\right] \Delta\dot{\mathbf{r}} + \mathbf{u}_r
\end{aligned} \tag{5}$$

Since the Lyapunov function is positive definite with a negative semidefinite derivative, the control law in Equation 5 is globally stabilizing. Thus, rate errors will converge to 0, however we can't yet guarantee that position errors will also converge to 0. To investigate position convergence, we can take further derivatives and apply the Mukherjee-Chen theorem [5]. Starting with the first derivative:

$$\begin{aligned}
\dot{V} = \Delta\dot{\mathbf{r}}^\intercal \left(\Delta\ddot{\mathbf{r}} + \left[K_1\right] \Delta\mathbf{r}\right) = -\Delta\dot{\mathbf{r}}^\intercal \left[K_2\right] \Delta\dot{\mathbf{r}} \implies \Delta\ddot{\mathbf{r}} + \left[K_1\right] \Delta\mathbf{r} &= \left[K_2\right] \Delta\dot{\mathbf{r}} \\
\downarrow, \Delta\dot{\mathbf{r}} = \mathbf{0} & \\
\Delta\ddot{\mathbf{r}} = -\left[K_1\right] \Delta\mathbf{r} &
\end{aligned} \tag{6}$$

Then the second derivative:

$$\begin{aligned}
\ddot{V} &= -2\Delta\ddot{\mathbf{r}}^\intercal \left[K_2\right] \Delta\dot{\mathbf{r}} \\
&\downarrow \\
\ddot{V}(\Delta\dot{\mathbf{r}} = \mathbf{0}) &\overset{\checkmark}{=} 0
\end{aligned}$$

Then the third derivative:

$$\begin{aligned}
\dddot{V} &= -2\Delta\dddot{\mathbf{r}}^\intercal \left[K_2\right] \Delta\dot{\mathbf{r}} - 2\Delta\ddot{\mathbf{r}}^\intercal \left[K_2\right] \Delta\ddot{\mathbf{r}} \\
&\downarrow \\
\dddot{V}(\Delta\dot{\mathbf{r}} = \mathbf{0}) &= -2\Delta\ddot{\mathbf{r}}^\intercal \left[K_2\right] \Delta\ddot{\mathbf{r}}
\end{aligned} \tag{7}$$

$$= -2\Delta\mathbf{r}^\intercal \left[K_1\right]^\intercal \left[K_2\right] \left[K_1\right] \Delta\mathbf{r} \overset{\checkmark}{<} 0 \tag{8}$$

Thus, since an odd derivative of the Lyapunov function is negative definite on the set where $\dot{V} = 0$, the control law is also asymptotically stabilizing.

With stability proven, the feedforward acceleration term in Equation 5, $\mathbf{u}_r$, needs to be defined. For any given ring, the reference position to put it in the proper race course configuration can be written as

$$\mathbf{r}_r = \mathbf{r}_c + \boldsymbol{\rho} \tag{9}$$

where $\boldsymbol{\rho} = {}^{O}[x, y, z]^\intercal$ is the ring's race course position in the Hill Frame. Taking the derivative of Equation 9:

$$
\begin{aligned}
\mathbf{v}_r = \dot{\mathbf{r}}_r &= \dot{\mathbf{r}}_c + x\dot{\hat{\mathbf{o}}}_r + y\dot{\hat{\mathbf{o}}}_\theta + z\dot{\hat{\mathbf{o}}}_h \\
&= \dot{\mathbf{r}}_c + x(\boldsymbol{\omega}_{O/\mathcal{N}} \times \hat{\mathbf{o}}_r) + y(\boldsymbol{\omega}_{O/\mathcal{N}} \times \hat{\mathbf{o}}_\theta) + z(\boldsymbol{\omega}_{O/\mathcal{N}} \times \hat{\mathbf{o}}_h) \\
&\downarrow, \boldsymbol{\omega}_{O/\mathcal{N}} = \dot{f}\hat{\mathbf{o}}_h \\
&= \dot{\mathbf{r}}_c + x\dot{f}\hat{\mathbf{o}}_\theta - y\dot{f}\hat{\mathbf{o}}_r
\end{aligned} \tag{10}
$$

Taking one more derivative, we can back out $\mathbf{u}_r$:

$$
\begin{aligned}
\ddot{\mathbf{r}}_r &= \ddot{\mathbf{r}}_c + (\dot{x}\dot{f} + x\ddot{f})\hat{\mathbf{o}}_r + x\dot{f}\dot{\hat{\mathbf{o}}}_\theta - (\dot{y}\dot{f} + y\ddot{f})\hat{\mathbf{o}}_r - y\dot{f}\dot{\hat{\mathbf{o}}}_r \\
&\downarrow, \dot{x} = \dot{y} = 0 \\
&= \ddot{\mathbf{r}}_c - (x\dot{f}^2 + y\ddot{f})\hat{\mathbf{o}}_r + (x\ddot{f} - y\dot{f}^2)\hat{\mathbf{o}}_\theta \\
&= \mathbf{f}(\mathbf{r}_r, \mathbf{v}_r) + \mathbf{u}_r \\
&\downarrow \\
\mathbf{u}_r &= \mathbf{f}(\mathbf{r}_c) - \mathbf{f}(\mathbf{r}_r, \mathbf{v}_r) - (x\dot{f}^2 + y\ddot{f})\hat{\mathbf{o}}_r + (x\ddot{f} - y\dot{f}^2)\hat{\mathbf{o}}_\theta
\end{aligned} \tag{11}
$$

Where $\mathbf{f}(\mathbf{r}_c)$ is the natural acceleration of the race course origin without any orbit perturbations, i.e. a fully Keplerian orbit. We can simplify Equation 11 using the following identities to get a final expression for feedforward acceleration:

$$
\begin{aligned}
& h = r_c^2 \dot{f} \\
& \dot{h} = 0 = 2r_c\dot{r}_c + r_c^2\ddot{f} \implies \ddot{f} = -2\frac{\dot{r}_c}{r_c}\dot{f} \\
& \quad \downarrow \\
\mathbf{u}_r &= \mathbf{f}(\mathbf{r}_c) - \mathbf{f}(\mathbf{r}_r, \mathbf{v}_r) + [\mathcal{NO}] \begin{bmatrix} -x\dot{f}^2 - 2y\frac{\dot{r}_c}{r_c}\dot{f} \\ -2x\frac{\dot{r}_c}{r_c}\dot{f} - y\dot{f}^2 \\ 0 \end{bmatrix} \\
&= \mathbf{f}(\mathbf{r}_c) - \mathbf{f}(\mathbf{r}_r, \mathbf{v}_r) - \dot{f}\,[\mathcal{NO}] \underbrace{\begin{bmatrix} \dot{f} & 2\frac{\dot{r}_c}{r_c} & 0 \\ 2\frac{\dot{r}_c}{r_c} & \dot{f} & 0 \\ 0 & 0 & 0 \end{bmatrix}}_{[A]} \boldsymbol{\rho} \\
&= \mathbf{f}(\mathbf{r}_c) - \mathbf{f}(\mathbf{r}_r, \mathbf{v}_r) - \dot{f}\,[\mathcal{NO}]\,[A]\,\boldsymbol{\rho}
\end{aligned} \tag{12}
$$

Finally, plugging Equation 12 into the control law from Equation 5 results in the final race course formation control law:

$$
\mathbf{u} = \mathbf{f}(\mathbf{r}_c) - \mathbf{f}(\mathbf{r}_d, \mathbf{v}_d) - [K_1]\,\Delta\mathbf{r} - [K_2]\,\Delta\dot{\mathbf{r}} - \dot{f}\,[\mathcal{NO}]\,[A]\,\boldsymbol{\rho} \tag{13}
$$

This controller can be easily tuned for performance with the PD-control-like gains $[K_1]$ and $[K_2]$.

B. Lead-Follower Formation Control

The goal of the lead-follower formation control law is to return the rings back to the idle configuration from the race course formation with as little effort as possible and without resorting to impulsive control laws. A lead-follower formation is a formation in which the only nonzero orbit element difference between the deputy and chief is mean anomaly. Thus, the easiest way to enforce that constraint is to control the orbit elements directly, and so an orbit element difference controller was chosen.

We know that the orbit elements of a satellite orbiting a celestial body are as follows, assuming the control sensitivity to first-order Brouwer-Lyddane theory is identity [4]:

$$\dot{\mathbf{oe}} \approx [A(\mathbf{oe})] + [B(\mathbf{oe})]\,\mathbf{u} \tag{14}$$

where, assuming $\mathbf{oe} = [a, e, i, \Omega, \omega, M]^\intercal$,

$$[A(\mathbf{oe})] = \begin{bmatrix} 0 \\ 0 \\ 0 \\ -\frac{3}{2}J_2\left(\frac{r_{eq}}{p}\right)^2 n\cos i \\ \frac{3}{4}J_2\left(\frac{r_{eq}}{p}\right)^2 n(5\cos^2 i - 1) \\ n + \frac{3}{4}J_2\left(\frac{r_{eq}}{p}\right)^2 \eta n(3\cos^2 i - 1) \end{bmatrix}$$

is the natural evolution of mean orbit elements under the influence of J_2 and

$$[B(\mathbf{oe})] = \begin{bmatrix} \frac{2a^2 e\sin f}{h r_{eq}} & \frac{2a^2 p}{h r r_{eq}} & 0 \\ \frac{p\sin f}{h} & \frac{(p+r)\cos f + re}{h} & 0 \\ 0 & 0 & \frac{r\cos\theta}{h} \\ 0 & 0 & \frac{r\sin\theta}{h\sin i} \\ -\frac{p\cos f}{he} & \frac{(p+r)\sin f}{he} & -\frac{r\sin\theta\cos i}{h\sin i} \\ \frac{\eta(p\cos f - 2re}{he} & -\frac{\eta(p+r)\sin f}{he} & 0 \end{bmatrix}$$

is the control to output matrix from Gauss's Variational Equations.

To derive the lead-follower formation control law, we will once again utilize Lyapunov's Direct Method. We can define an error vector $\Delta\mathbf{oe} = \mathbf{oe}_d - \mathbf{oe}_r$ and propose the following positive definite, radially unbounded candidate Lyapunov function:

$$V(\Delta\mathbf{oe}) = \frac{1}{2}\Delta\mathbf{oe}^\intercal\,[K]\,\Delta\mathbf{oe} \tag{15}$$

where $[K]$ is a symmetric positive definite matrix. We can then take the derivative of Equation 15 as follows:

$$\begin{aligned} \dot{V} &= \Delta\mathbf{oe}^\intercal\,[K]\,\Delta\dot{\mathbf{oe}} \\ &= \Delta\mathbf{oe}^\intercal\,[K]\,(\dot{\mathbf{oe}}_d - \dot{\mathbf{oe}}_r) \end{aligned} \tag{16}$$

A lead-follower formation is naturally occurring, so we know that $\dot{\mathbf{oe}}_r = [A(\mathbf{oe}_r)]$ and there is no need for feedforward control. Plugging into Equation 16:

$$\dot{V} = \Delta\mathbf{oe}^\intercal\,[K]\,([A(\mathbf{oe}_d)] - [A(\mathbf{oe}_r)] + [B(\mathbf{oe}_d)]\,\mathbf{u}) \tag{17}$$

We can further simplify Equation 17 by realizing that $\mathbf{oe}_d \approx \mathbf{oe}_r$ for most of the scenario. This can be reasoned by noticing that the maximum deployed ring distance from the race course origin is on the order of 1-3 km, compared to the course origin orbit's semi-major axis of 10,566 km for a 3 hour orbit. Thus, the contribution from the difference of $[A(\mathbf{oe})]$ terms is small and can be dropped without making too much of an impact. This assumes that the course origin is now under the influence of J_2, which is acceptable since the lead-follower formation doesn't necessarily need the course origin to have a Keplerian orbit. This is different from the race course formation, in which making the course origin orbit Keplerian was useful for defining the race course geometry. We can do this because the course origin is just defined as a point of reference for the rest of the race course, and hence not necessarily subject to consistent physical

constraints. This would not be true of a chief that is an actual spacecraft, but that is not the case here. All of that being said, the derivative of the candidate Lyapunov function then becomes:

$$\dot{V} \approx \Delta\boldsymbol{\alpha}\mathbf{e}^{\intercal}\ [K]\ [B(\boldsymbol{\alpha}\mathbf{e}_d)]\,\mathbf{u} \tag{18}$$

We can force this expression to be negative definite by intelligently choosing the control law

$$\mathbf{u} = -\,[B(\boldsymbol{\alpha}\mathbf{e}_d)]^{\intercal}\ [K]\,\Delta\boldsymbol{\alpha}\mathbf{e} \tag{19}$$

The control law in Equation 19 allows for tuning of the closed loop response to individual orbit element differences, which is useful for focusing on specific elements such as semi-major axis for formation boundedness and inclination for out of plane motion. Plugging the control law into Equation 18 proves that this control is also asymptotically stable, with a negative definite first derivative of the candidate Lyapunov function in terms of all the relevant states being controlled:

$$\dot{V}(\Delta\boldsymbol{\alpha}\mathbf{e}) = -\Delta\boldsymbol{\alpha}\mathbf{e}^{\intercal}\ [K]\ [B(\boldsymbol{\alpha}\mathbf{e}_d)]\ [B(\boldsymbol{\alpha}\mathbf{e}_d)]^{\intercal}\ [K]\,\Delta\boldsymbol{\alpha}\mathbf{e} \overset{\checkmark}{<} 0 \tag{20}$$

C. Controller Testing

With both controllers designed, they needed to be tuned. To do so, a single ring was chosen to be deployed to its race course location and then stowed back to the lead follower formation.

1. Race Course Controller Tuning

To tune the race course controller, a process similar to PD tuning was used. Namely, the ideal race course controller would deploy each ring quickly and with minimal overshoot. This was accomplished by adjusting gains until eventually choosing $[K_1]$ = 2e-5 and $[K_2]$ = 5e-3. Figures 5 and 6 show the effort expended to deploy the ring on a logarithmic scale and the resulting trajectory respectively.

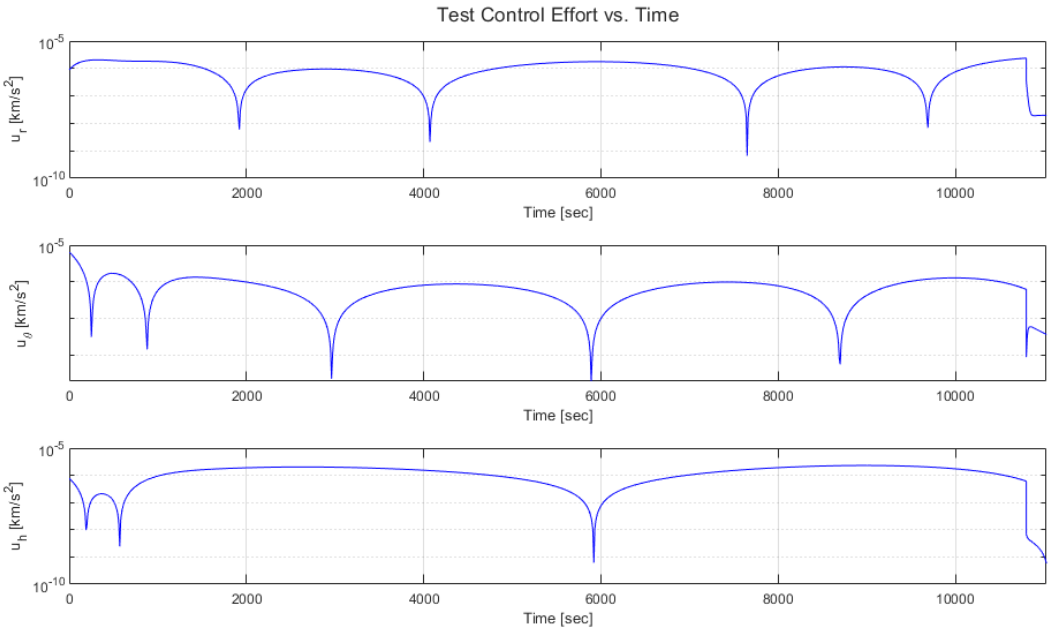

Fig. 5 Effort to Deploy Race Course Ring

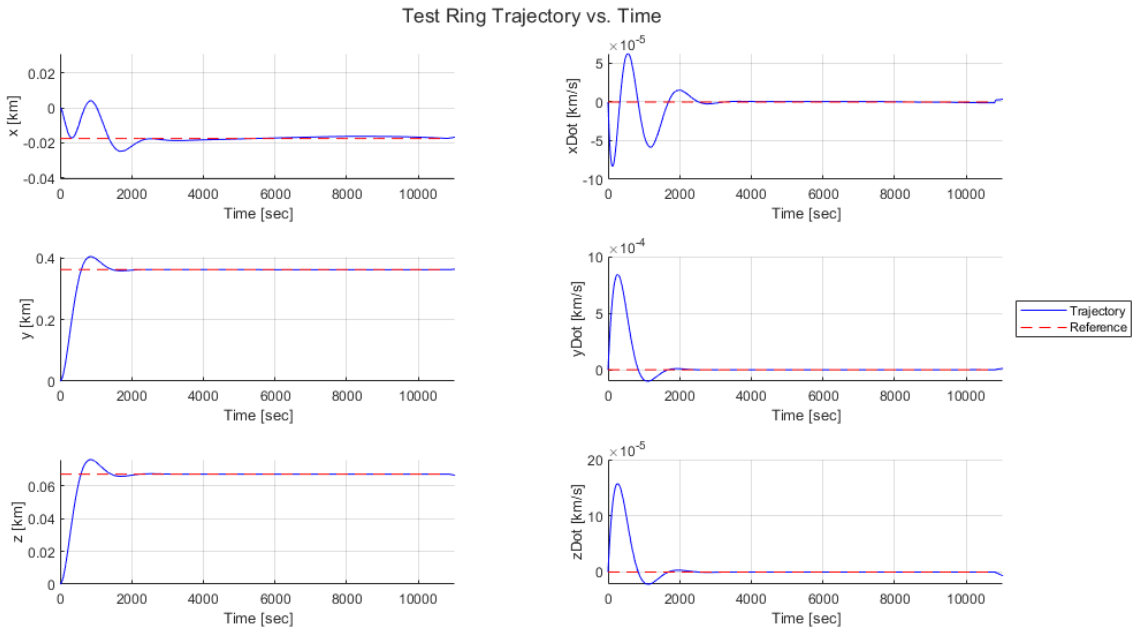

Fig. 6 Deployed Race Course Ring Trajectory

Figure 5 shows a decently consistent control effort on the order of 2e-6 km/s^2 throughout deployment, and Figure 6 shows a well-damped response that converges to the reference well within 1 chief orbit of 3 hours.

2. Lead-Follower Controller Tuning

To tune the lead-follower controller, it was determined that semi-major axis, inclination, and RAAN were the most important elements to control, followed by the others. Thus, the former were given twice the gain of the latter. After adjusting gains, the $[K]$ matrix was tuned to be

$$[K] = \begin{bmatrix} 8\text{e-}3 & 0 & 0 & 0 & 0 & 0 \\ 0 & 4\text{e-}3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 8\text{e-}3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 8\text{e-}3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 4\text{e-}3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 4\text{e-}3 \end{bmatrix}$$

Figures 7, 8, and 9 show the effort expended to stow the ring on a logarithmic scale, the resulting change in orbit elements, and resulting trajectory respectively.

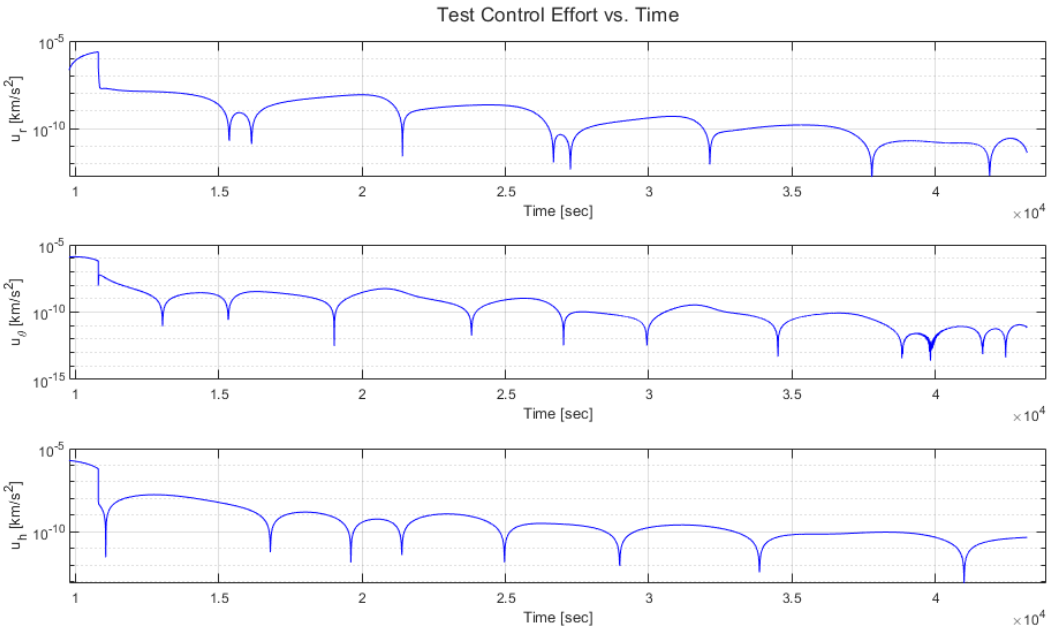

Fig. 7 Effort to Stow Race Course Ring

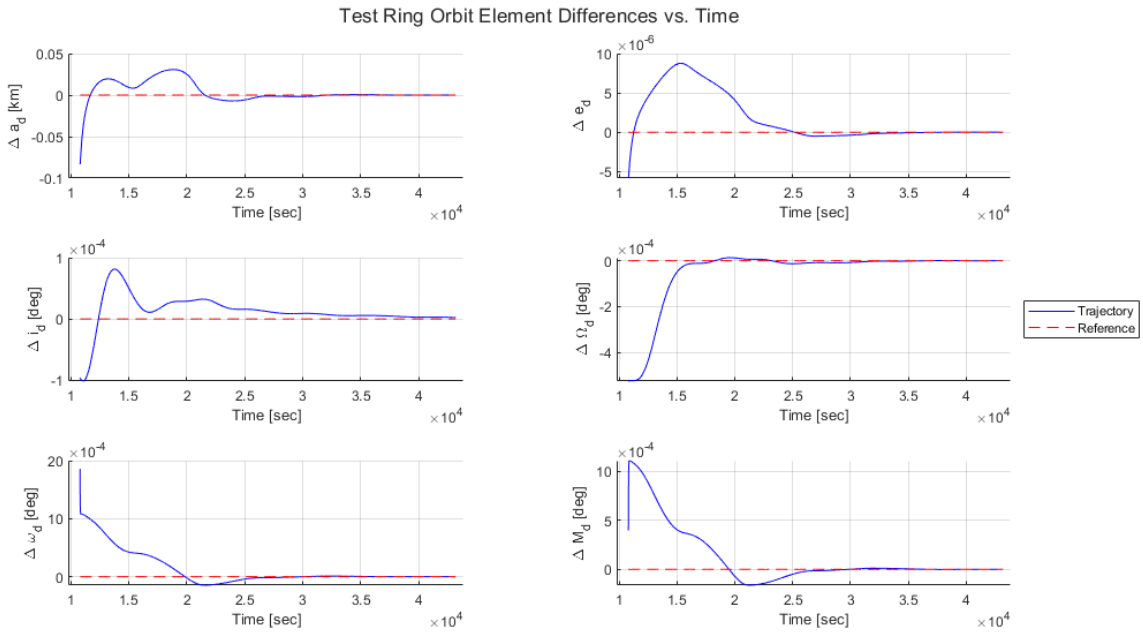

Fig. 8 Stow Race Course Ring Orbital Element Evolution

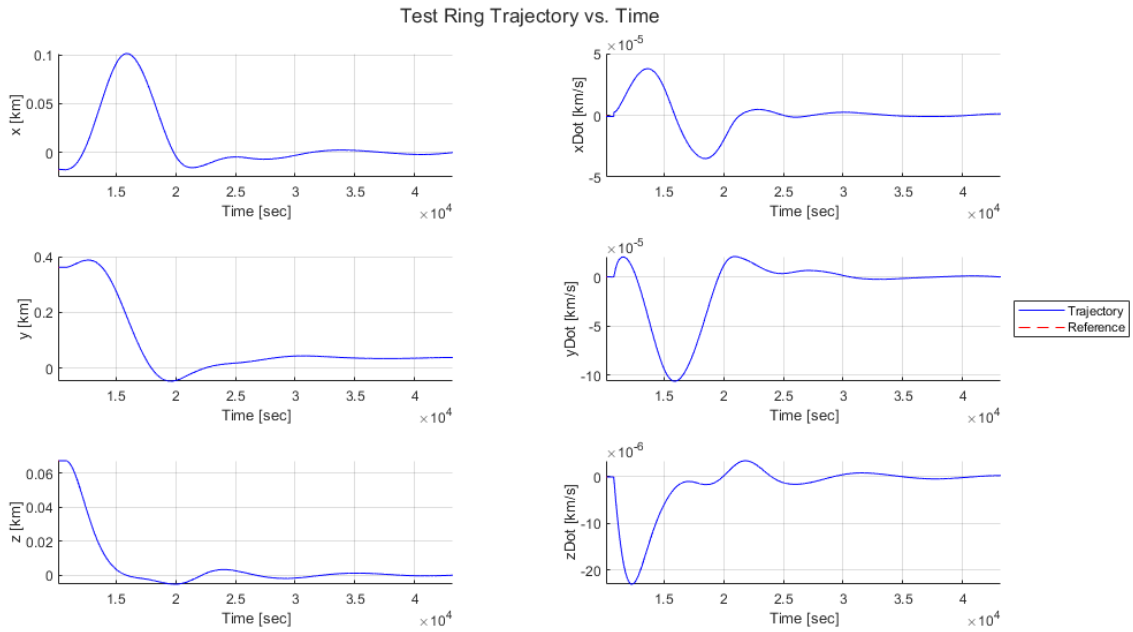

Fig. 9 Stowed Race Course Ring Trajectory

Figure 7 shows a variable control effort on the order of 1e-8 to 1e-9 km/s^2, which is about 200-2000 times less effort than the race course deployment controller. Eventually, the control effort flattens out to about 2e-12 km/s^2, which implies that the natural lead-follower formation has been achieved. Figure 8 confirms this fact, showing orbit element differences that all converge to 0 over the course of roughly 2.5 chief orbits. Figure 9 shows that the resulting trajectory is smooth and bounded, eventually also settling to the lead-follower formation after about 2.5 orbits.

V. Tasks 2-3: Ring Deployment and Stowing Sequence Development

With the formation controllers designed, a sequence to deploy and stow the rings was required.

A. Race Course Deployment Sequence

An ideal ring deployment sequence would safely and efficiently deploy rings from the idle lead-follower formation to the active race course formation. To do this, rings were sorted based on the distance of their race course location to the race course origin. Then, rings are deployed in order of decreasing distance to the origin, starting with the start and end race course rings. For example, the start of the sequence would be

1) End ring
2) Start ring
3) Farthest race course ring
4) Second farthest race course ring
5) etc.

Once rings are sorted, they are also assigned an initial mean anomaly around the course origin such that rings are evenly spread ahead of and behind the origin. The deployment sequence can be constructed by running Algorithm 1:

Algorithm 1 Ring Deployment Sequence Construction (rings)

Require: $rings$ is an array of ring structures each containing the precomputed race course location for that ring, with the start ring as the first ring in the array and the end ring as the last ring in the array.
Define empty $sortedDistances$ and $deploymentOrder$ arrays
for i from 2 to size($rings$) − 1 **do**
Calculate distance of ring i to the race course origin
end for
Sort distances from the origin in ascending order and store in $sortedDistances$
for i from 1 to size($sortedDistances$) **do**
Set all desired orbit elements to 0
if i is odd **then**
Set desired mean anomaly to i*(1e-4) + 0.5e-4 degrees
Convert to radians
else
Set desired mean anomaly to i*(-1e-4) + 0.5e-4 degrees
Convert to radians
end if
Convert desired orbit elements to cartesian and assign ring initial state
Append $rings(i)$.ringNumber to $deploymentOrder$
end for
Add start ring -4e-4 degrees behind the minimum mean anomaly, convert to radians, and calculate initial state.
Append 1 to $deploymentOrder$.
Add end ring 4e-4 degrees ahead of the maximum mean anomaly, convert to radians, and calculate initial state.
Append size($rings$) to $deploymentOrder$.
Flip $deploymentOrder$ so start and end rings are deployed first, followed by decreasing distances to the origin.
return $deploymentOrder$

Running this procedure on the generated race course in Figure 4 results in Table 2:

Table 2 Race Course Ring Deployment

Ring Number	Deployed Distance to Origin [km]	Deployment Slot	Initial Mean Anomaly [deg]
1	1.1845	2	-0.00155
2	1.1421	4	-0.00115
3	1.1923	3	0.00135
4	0.8873	8	-0.00075
5	0.6327	10	-0.00055
6	0.5782	12	-0.00035
7	0.3684	13	0.00035
8	0.0847	15	0.00015
9	0.3475	14	-0.00015
10	0.6221	11	0.00055
11	0.8379	9	0.00075
12	1.0241	6	-0.00095
13	1.1221	5	0.00115
14	1.0086	7	0.00095
15	0.8501	1	0.00175

Thus, the rings would be deployed in the following order, with 500 seconds in between each deployment: 15 (end ring), 1 (start ring), 3 (furthest ring), 2 (next furthest ring), 13, 12, and so on until ring 8 (closest ring).

B. Race Course Stowing Sequence

To stow the race course formation, we can simply follow the deployment order in reverse, as the ideal stowing sequence would stow closer rings first and then progressively further ones. Since the lead-follower controller is much slower than the race course controller, each ring will be stowed in 1000 second intervals instead of 500 second intervals. Thus, rings would be stowed in the following order: 8 (closest ring), 9 (next closest ring), ..., 3 (furthest ring), 1 (start ring), 15 (end ring).

VI. Task 4: Demonstration of the Full Deploy - Sustain - Stow Sequence

With the deployment and stowing sequences defined, the entire scenario can now be run. For full visualization benefits, it is recommended to view the rendered animation linked in Appendix C.

In this scenario, the race course rings start out in the lead-follower formation for 1 orbit of the race course origin. After 1 orbit, the deployment sequence executes and holds the race course formation for 2 chief orbits. Then the stowing sequence executes over 3 chief orbits, with the lead-follower formation reforming 4 orbits later. In total, 10 chief orbits are simulated, i.e. 30 hours or 108000 seconds. Figures 10, 11, and 12 show the control effort, orbit element evolutions, and ring trajectories respectively over the scenario.

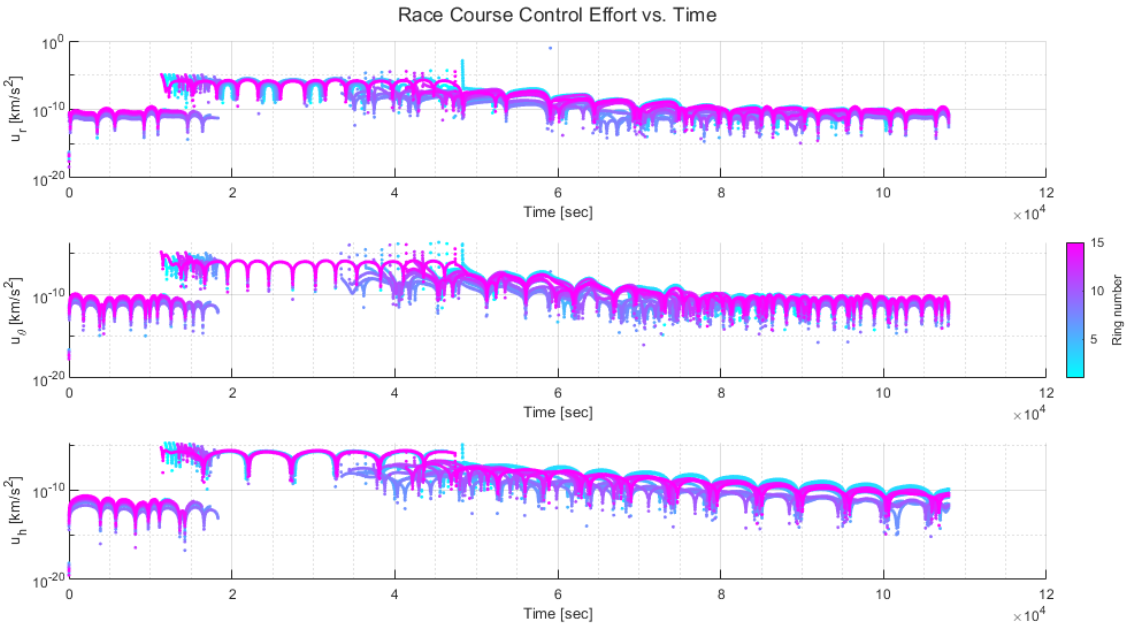

Fig. 10 Control Effort over the Deploy - Sustain - Stow Sequence

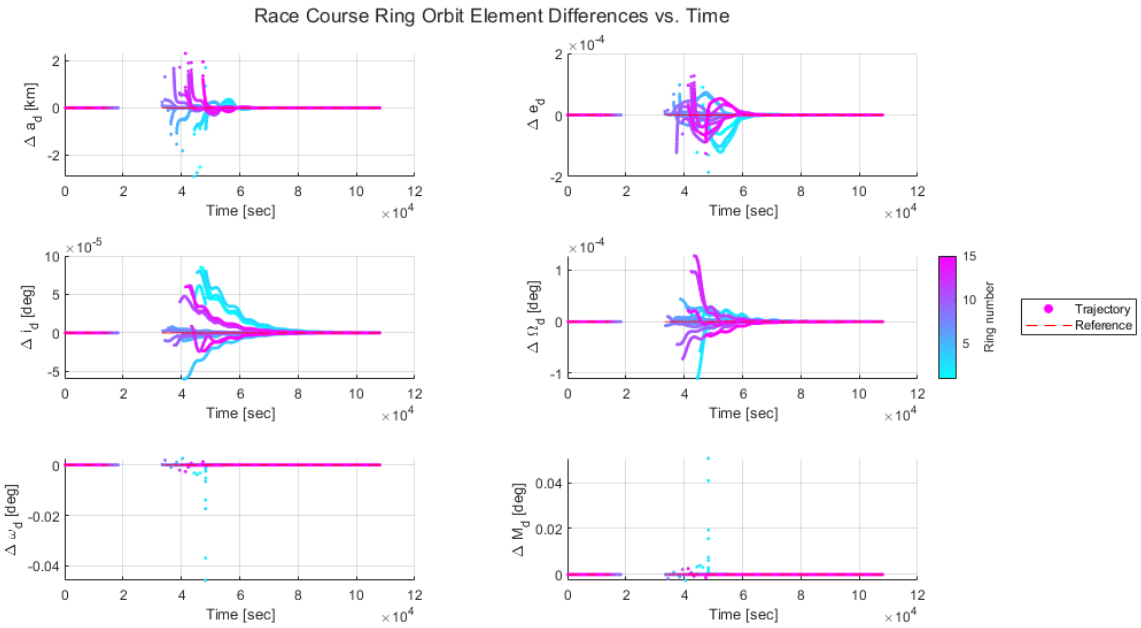

Fig. 11 Orbital Element Evolution over the Deploy - Sustain - Stow Sequence

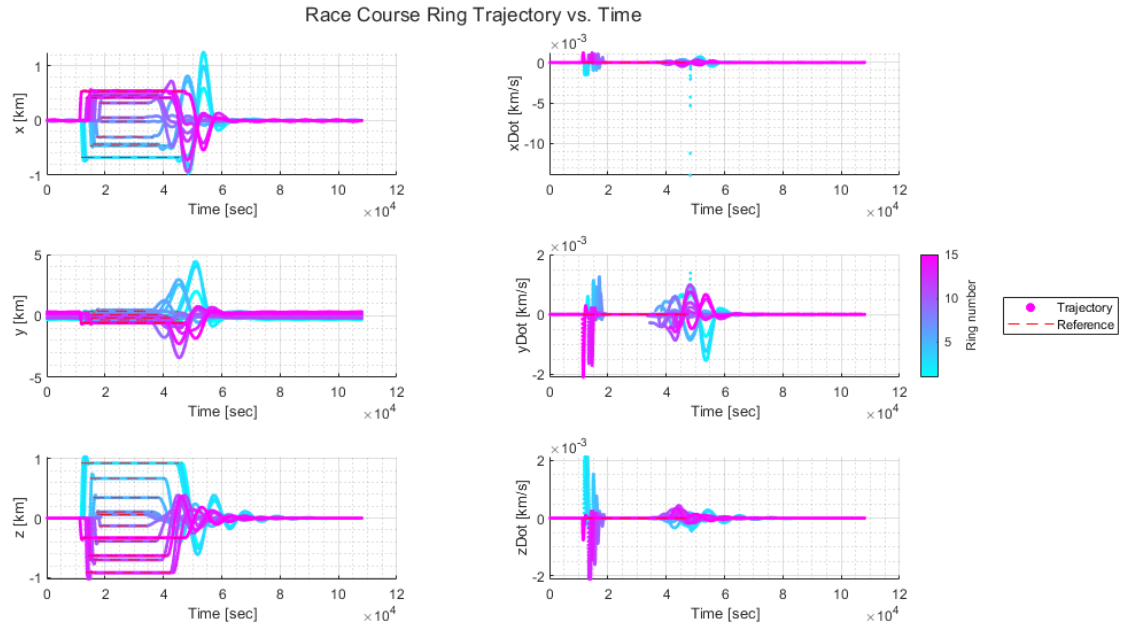

Fig. 12 Ring Trajectories over the Deploy - Sustain - Stow Sequence

Starting with control effort in Figure 10, we can see that the lead-follower formation is much cheaper to maintain than the full race course formation. In particular, we see an average control effort to sustain it on the order of 3e-10 to 5e-11 km/s^2 when the orbit element differences converge and on the order of 1e-8 to 1e-9 km/s^2 when converging. This is much cheaper than the active race course formation control, which has relatively constant control efforts on the order of 2e-6 km/s^2.

Moving onto orbital elements in Figure 11, we can see that all errors converge to 0 within 4 chief orbits after the

race course formation is transitioned to the lead-follower formation. There is a small numerical artifact in $\Delta\omega_d$ and ΔM_d about 4.5 chief orbits in, which is related to M_d and M_r wrapping from π to $-\pi$ about one timestep apart. This causes a momentary spike in the error vector, which then commands a large acceleration to compensate. This can be mitigated by unwrapping M to make it a continuous measurement rather than a periodic one, which would get rid of the periodic spikes between π and $-\pi$.

Finally, we can see a very clear distinction between each formation in Figure 12, where we see each ring deploy to its specified cartesian position and hold before the trajectory oscillates down to achieve the lead-follower formation. We can also see the deployment and stowing sequence in action by noticing the staggered step response-like evolution of the cartesian ring positions. We can also see that the ring velocities are generally slower in the stowing sequence than in the deployment sequence, albeit they oscillate much longer. In that vein, we also see the positions balloon to far outside the race course bounds in the along-track direction, while generally staying in bounds in the radial and cross-track directions. This is an area of improvement that could be investigated in a future project.

VII. Conclusion

To conclude, two race course ring formation control strategies were designed and implemented: one for the race course formation itself, and one for the idle lead-follower formation. Both controllers are asymptotically stabilizing and perform well, and should work for any random race course - not just the one presented here. In addition, deployment and stowing sequences were designed to intelligently switch between the controllers and take the formation from the lead-follower configuration to the race course configuration and back.

Outside of CubeSat racing, the controllers developed here are generally useful. The race course formation controller is essentially a controller that takes a satellite from any initial cartesian state to any other stationary cartesian state, which has implications for many different applications such as stationkeeping, inspection, and more. On the other hand, the lead-follower controller is a controller that takes any satellite state and forces it to a lead follower formation, which is a naturally bounded formation. Lead-followers are incredibly useful for all sorts of missions, such as NASA's A-train formation [6], which is a lead-follower formation of Earth-observing satellites that collect data on Earth's climate, atmosphere, and much more. A controller that takes a satellite from any cartesian state to a bounded lead-follower with minimal control effort is critical for risk mitigation and mission safety.

However, the controllers are not without fault. In the context of CubeSat racing, the lead-follower controller results in trajectories that could cause rings to collide. In the animation linked in Appendix C, we can see that many of the rings fly directly through the chief in the along track direction, irrespective of any rings that have already parked in their lead-follower slot. This is obviously a safety issue, and one that could be mitigated by modifying the stowing sequence. For example, the race course formation controller could be utilized to get the rings as close as possible to their parking slot, and then switch over to the lead-follower controller to avoid egregious overshoot from each ring. Another option would be to wait longer for each ring to stow, for example 2000 seconds rather than 1000. This would provide more time for each ring to settle into its parking slot before the next ring attempts to stow, but doesn't solve the overshoot problem.

In all, this project was a resounding success and would be well suited for implementation into a video game or some other fun application!

VIII. Acknowledgments

Special thanks to Dr. Scheeres for initially approving this deceptively fascinating project in the Optimal Trajectories course at CU Boulder, as well as to Dr. Schaub for approving me to continue to investigate it in the Spacecraft Formation Flying course at CU Boulder.

References

[1] Faber, I., "NASCAR for Cubesats: An Application of Primer Vector Theory," , 2025. URL `https://github.com/EpicIan60142/NASCARforCubeSats`.

[2] Lawden, D. F., *Analytical Methods of Optimization*, 1st ed., Dover Publications Inc., 2006.

[3] Faber, I., "NASCAR for Cubesats: Course Ring Relative Motion," , 2025. URL `https://drive.google.com/file/d/1RoqR9pxoQsUXqdwpru161aWAecso-chA/view?usp=sharing`.

[4] Schaub, H., and Junkins, J. L., *Analytical Mechanics of Space Systems*, 4th ed., AIAA, 2018.

[5] Mukherjee, R., and Chen, D., "Asymptotic Stability Theorem for Autonomous Systems," *Journal of Guidance, Control and Dynamics*, Vol. 16, 1993, pp. 961–963.

[6] NASA, "Formation Flying," , 2024. URL `https://atrain.nasa.gov/`.

A. Appendix Index

Use the appendix index below to navigate to the different appendices!

B. Controller Code

Back to appendix index: A.A

B. Race Course Formation Controller

Back to code index: B.A

```
function [dX, u] = ringFormationFeedbackControl(t, X, rho, kConst, pConst,
    scConst)
% Function that implements an inertial feedback control law for race course
% ring formation control, feeding forward for a desired ring center from
% the center of the race course
%   Inputs:
%       - t: Current integration time in sec
%       - X: Current combined state vector as follows:
%            X = [X_c; X_d]
%       - rho: Desired ring center position in the Hill Frame like so:
%              rho = [x; y; z]
%       - kConst: Structure of control gains with K1 and K2 as fields
%       - pConst: Planetary constants vector containing mu for the
%                 celestial body of interest
%       - scConst: Ring structure containing drag parameters Cd, m, and A
%   Outputs:
%       - dX: Rate of change vector as follows:
%             [dX_c; dX_d]
%       - u: Control in the body frame: [u_r; u_theta; u_h]
%
%   By: Ian Faber, 11/19/2025

% Extract states
X_c = X(1:6);
X_d = X(7:12);

% Calculate intermediate variables
rChief = X_c(1:3);
vChief = X_c(4:6);
h = norm(cross(rChief, vChief));
fDot = h/(norm(rChief)^2);

rHat = rChief/norm(rChief);
hHat = cross(rChief, vChief)/h;
```

```
thetaHat = cross(hHat, rHat);

NH = [rHat, thetaHat, hHat];

r_c = norm(rChief);
rDot_c = dot(NH'*vChief, [1;0;0]); % Radius rate of change

% Calculate reference inertial state
x = rho(1);
y = rho(2);

r_r = rChief + NH*rho;
v_r = vChief -y*fDot*rHat + x*fDot*thetaHat;

X_r = [r_r; v_r];

% Calculate error vector
dX_d = X_d - X_r;

% Calculate control effort
    % Gains
K1 = kConst.K1;
K2 = kConst.K2;

    % Natural accel
fChief = orbitEOM(t, X_c, pConst, struct(), false(1,2));
fChief = fChief(4:6);
fDeputy = orbitEOM(t, X_d, pConst, scConst, true(1,2));
fDeputy = fDeputy(4:6);
fDesired = orbitEOM(t, X_r, pConst, scConst, true(1,2));
fDesired = fDesired(4:6);

    % Feedforward
A = [
        fDot,           2*rDot_c/r_c,   0;
        2*rDot_c/r_c,   fDot,           0;
        0,              0,              0
    ];
u_d = fChief - fDesired - fDot*NH*A*rho;

    % Control law
u = -(fDeputy - fDesired) - K1*dX_d(1:3) - K2*dX_d(4:6) + u_d;

% Assign output
dX_c = [X_c(4:6); fChief];
dX_d = [X_d(4:6); fDeputy + u];

dX = [dX_c; dX_d];

% Report control in body frame
u = NH'*u;

end
```

C. Lead-Follower Formation Controller

Back to code index: B.A

```
function [dX, u, doe, oe_d, oe_c, oe_r] = leadFollowerFeedbackControl(t, X,
    doe_r, kConst, pConst, scConst)
% Function that implements an orbit element feedback control law for
% relative motion between a chief and deputy spacecraft, with desired
% motion specified in orbit element differences.
%   Inputs:
%       - t: Current integration time in sec
%       - X: Current combined state vector as follows:
%            X = [X_c; X_d]
%       - doe_r: Structure of desired orbit element differences with any of
%                the following fields, not necessarily all:
%                   da, de, di, dRAAN, dargPeri, dM0
%                If a field is missing, it is ignored and assumed not
%                desirable to control.
%       - kConst: Structure of control gains with the following fields:
%                 - P11, P22, P33: control gains
%       - pConst: Planetary constants vector containing mu for the
%                 celestial body of interest
%   Outputs:
%       - dX: Rate of change vector as follows:
%             [dX_c; dX_d]
%       - u: Control effort for the given time and state
%       - doe: Controller orbit element difference vector at the given time
%       - oe_d: Deputy orbit elements for the given time and state
%       - oe_c: Chief orbit elements for the given time and state
%       - oe_r: Reference orbit elements
%
%   By: Ian Faber, 11/20/2025

% Extract states
X_c = X(1:6);
X_d = X(7:12);

% Calculate state elements
cart.rVec = X_c(1:3);
cart.vVec = X_c(4:6);
cart.mu = pConst.mu;
oe_c = convCart2ClassicOE(cart);

cart.rVec = X_d(1:3);
cart.vVec = X_d(4:6);
cart.mu = pConst.mu;
oe_d = convCart2ClassicOE(cart);

rDeputy = cart.rVec;
vDeputy = cart.vVec;

% Calculate DCM
rHat = rDeputy/norm(rDeputy);
hHat = cross(rDeputy, vDeputy)/norm(cross(rDeputy, vDeputy));
thetaHat = cross(hHat, rHat);
```

```
51
52  NHd = [rHat, thetaHat, hHat];
53
54  % Calculate reference elements and populate B matrix and difference vector
55  B = [];
56  doe = [];
57  h = norm(cross(X_d(1:3), X_d(4:6)));
58  r_e = 6378; % km, radius of Earth
59  r = norm(X_d(1:3));
60  p = oe_d.a*(1-oe_d.e^2);
61  eta = sqrt(1-oe_d.e^2);
62
63  oe_r.mu = pConst.mu;
64  try doe_r.da;
65      oe_r.a = oe_c.a + doe_r.da;
66      Belem = [(2*oe_d.a^2*oe_d.e*sin(oe_d.f))/(h*r_e), (2*oe_d.a^2*p)/(h*r*r_e)
            , 0];
67      B = [B; Belem];
68      doe = [doe; oe_d.a/r_e - oe_r.a/r_e];
69  end
70
71  try doe_r.de;
72      oe_r.e = oe_c.e + doe_r.de;
73      Belem = [(p*sin(oe_d.f))/h, ((p+r)*cos(oe_d.f) + r*oe_d.e)/h, 0];
74      B = [B; Belem];
75      doe = [doe; oe_d.e - oe_r.e];
76  end
77
78  try doe_r.di;
79      oe_r.i = oe_c.i + doe_r.di;
80      Belem = [0, 0, (r*cos(oe_d.argPeri + oe_d.f))/h];
81      B = [B; Belem];
82      doe = [doe; oe_d.i - oe_r.i];
83  end
84
85  try doe_r.dRAAN;
86      oe_r.RAAN = oe_c.RAAN + doe_r.dRAAN;
87      Belem = [0, 0, (r*sin(oe_d.argPeri + oe_d.f))/(h*sin(oe_d.i))];
88      B = [B; Belem];
89      doe = [doe; oe_d.RAAN - oe_r.RAAN];
90  end
91
92  try doe_r.dargPeri;
93      oe_r.argPeri = oe_c.argPeri + doe_r.dargPeri;
94      Belem = [-(p*cos(oe_d.f))/(h*oe_d.e), ((p+r)*sin(oe_d.f))/(h*oe_d.e), -(r*
            sin(oe_d.argPeri + oe_d.f)*cos(oe_d.i))/(h*sin(oe_d.i))];
95      B = [B; Belem];
96      doe = [doe; oe_d.argPeri - oe_r.argPeri];
97  end
98
99  try doe_r.dM0;
100     M_c = convE2M(convf2E(oe_c.f, oe_c.e), oe_c.e);
101     M_r = M_c + doe_r.dM0;
102     oe_r.f = convE2f(convM2E(M_r, oe_r.e, false), oe_r.e);
```

```
    M_d = convE2M(convf2E(oe_d.f, oe_d.e), oe_d.e);
    Belem = [(eta*(p*cos(oe_d.f)-2*r*oe_d.e))/(h*oe_d.e), -(eta*(p+r)*sin(oe_d
        .f))/(h*oe_d.e), 0];
    B = [B; Belem];
    doe = [doe; M_d - M_r];
end

% Assign gains
% P = diag([kConst.P11, kConst.P22, kConst.P33]);
K = diag([kConst.K11, kConst.K22, kConst.K33, kConst.K44, kConst.K55, kConst.
    K66]);

% Calculate control effort and convert to inertial frame
if oe_d.e > 0.999
    u = zeros(3,1);
else
    % u = -P*B'*doe;
    u = -B'*K*doe;

    u = NHd*u;
end

% Assign output
fChief = orbitEOM(t, X_c, pConst, struct(), [true,false]);
fChief = fChief(4:6);

fDeputy = orbitEOM(t, X_d, pConst, scConst, true(1,2));
fDeputy = fDeputy(4:6);

dX_c = [X_c(4:6); fChief];
dX_d = [X_d(4:6); fDeputy + u];

dX = [dX_c; dX_d];

% Report control in body frame
u = NHd'*u;

end
```

C. Animations

Back to appendix index: A.A
Back to sequence demonstration: VI

To access animations for each Scenario in this report, click the associated Google Drive link!

- **Controller Testing**:
  https://drive.google.com/file/d/1g0l-UKKZkmcAqtpVmggVQuIr0bEO2UoF/view?usp=sharing
- **Full Deployment - Sustain - Stow Sequence**:
  https://drive.google.com/file/d/1ZtNN5Rfu15oJZL7Nqj2tqPSEyEXJ2uhS/view?usp=sharing

Enjoy!